Transport Layer – II

Transmission Control Protocol and User Datagram Protocol

Transmission Control Protocol (TCP)

- TCP was specifically designed to provide a reliable, end-to-end byte stream over an unreliable **internetwork**.

The problem TCP solves

- An internetwork joins parts with widely different topologies, bandwidths, delays and packet sizes
- No path parameter can be fixed in advance, so TCP adjusts its timers and its sending rate from what it observes on the connection
- It is designed to remain correct in the face of loss, duplication, reordering and host failure

Core properties

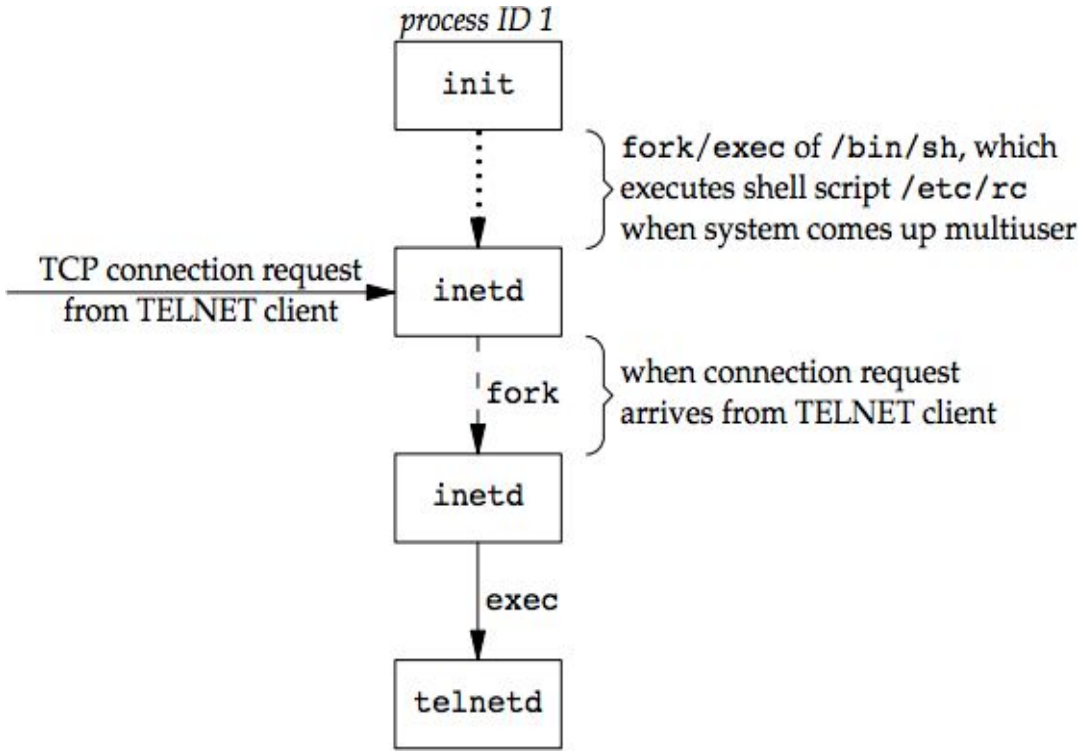
- Connection oriented: state is established before data flows and released afterwards
- Byte stream: the unit is the byte, not the message
- Full duplex: data flows both ways on one connection
- Point to point: exactly two endpoints, so no multicast or broadcast

Specification

- RFC 9293 (2022) is the current base specification. It obsoletes RFC 793 (1981) and gathers the accumulated Standards Track changes into one document
- RFC 7323 (2014): window scale, timestamps, PAWS. Obsoletes RFC 1323
- RFC 2018: selective acknowledgement (SACK)
- RFC 5681: congestion control. Obsoletes RFC 2581
- RFC 3168: explicit congestion notification (ECN)

TCP Service Model

- Uses **Sockets** to define an end-to-end connection (Source IP, Source Port, Destination IP, Destination Port). A socket identifies an endpoint.
- A pair of sockets identifies a **connection.**
- Including the protocol gives the 5-tuple that identifies the flow uniquely across all transport protocols
- A server binds to a well-known port and accepts connections on it.
- Each accepted connection becomes a separate socket pair, with its own state, sequence numbers and buffers.

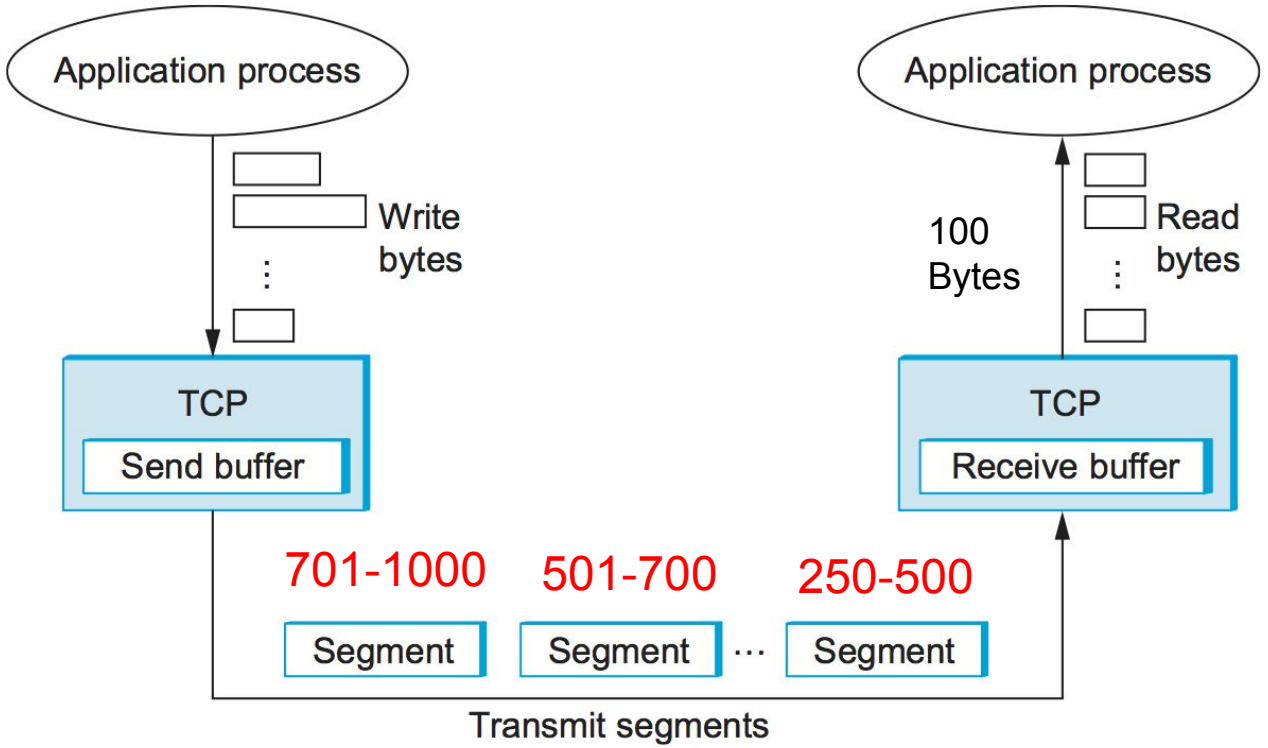


All TCP connections are full-duplex and point-to-point. TCP does not support multicasting or broadcasting

TCP Service Model

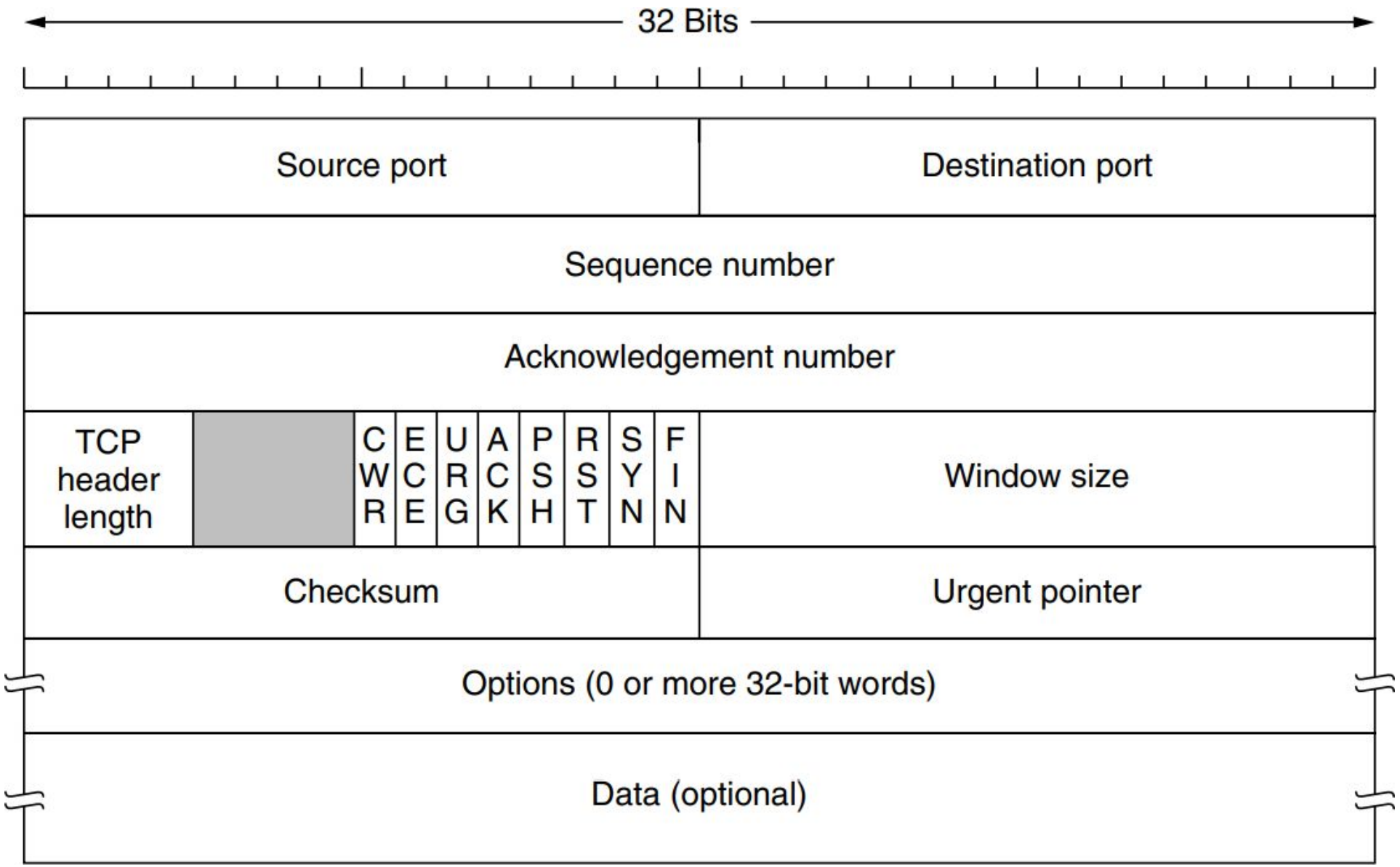
- A TCP connection is a **byte stream**, not a message stream
- Message boundaries are *not preserved end-to-end*

Variable size bytes streams



There is no way for the receiver to detect the unit(s) in which the data were written by the sending process.

TCP Protocol- Header



TCP Header

- Every segment begins with a fixed-format, 20-byte header.
- The fixed header may be followed by header options (max 40 bytes).
- After the options, if any, up to 65,535 – 20 – 20 = 65,495 data bytes may follow, where the first 20 refer to the IP header and the second to the TCP header.
- In practice the segment size is limited by the path MTU, not by this figure.
- Segments without any data are legal and are commonly used for acknowledgements and control messages.

TCP Header: Port Numbers

The fields

- Source port and destination port, 16 bits each
- They identify the two end points of the connection, one on each host
- 16 bits gives 65,536 port numbers per host, per transport protocol

Port ranges (IANA, RFC 6335)

- System ports, 0 to 1023: well-known services, assigned by IANA
- User ports, 1024 to 49151: registered services, assigned on request
- Dynamic ports, 49152 to 65535: never assigned, used by clients as temporary source ports

From port to connection

- A port number together with its host's IP address forms a socket. In IPv4 this is a 48-bit end point
- The two sockets together identify the connection
- Written out, the connection identifier is the 5-tuple: <protocol, source IP, source port, destination IP, destination port>

Why all five values matter

- TCP demultiplexes an arriving segment using all of them, not the destination alone
- So one server port supports many simultaneous connections, one per distinct client socket
- Two connections from the same client differ in their source ports and are therefore distinct

TCP Sequence Number and Acknowledgement Number

Both fields are 32 bits, and both count bytes, not segments.

Sequence number

- Every byte on a TCP connection has its own 32-bit sequence number
- The sequence number in a segment's header is that of the **first data byte** the segment carries
- Numbering starts from an initial sequence number chosen at connection setup, not from zero

Acknowledgement number

- Carries the **next byte expected**, in order
- It is cumulative: it acknowledges every byte below it, received correctly and in sequence
- Valid only when the ACK bit is set, which is the case for nearly all segments

Example

- ACK = 31245 means every byte up to and including sequence number 31244 has been received correctly, and byte 31245 is expected next

TCP Segments

TCP chooses its own segment boundaries. The application's writes do not determine them.

What a segment is

- The sending and receiving TCP entities exchange data in the form of segments
- A segment is a 20-byte fixed header, plus options if present, followed by zero or more data bytes

TCP controls the boundaries

- It can accumulate data from several write() calls into one segment
- It can split the data from one write() across several segments
- This is the mechanism behind the byte-stream service: boundaries are TCP's to choose, not the application's

Two limits on segment size

- **Absolute:** the IP payload is at most 65,515 bytes (65,535 minus the 20-byte IP header), leaving at most 65,495 data bytes once the TCP header is subtracted
- **Practical:** the MTU of the path. A segment must fit in one IP packet without fragmentation, so the smallest MTU along the path governs

TCP Header Length and Control Bits

Header length

- The number of 32-bit words in the TCP header
- Needed because the Options field is variable, so the header is too
- 4 bits, so the header is 5 to 15 words, that is 20 to 60 bytes

The eight control bits

Bit	Meaning
CWR	Congestion Window Reduced: the sender has slowed down
ECE	ECN-Echo: tells the sender to slow down
URG	The Urgent pointer field is in use
ACK	The Acknowledgement number field is valid
PSH	Deliver this data to the application on arrival
RST	Reset the connection
SYN	Synchronize sequence numbers, used to establish a connection
FIN	The sender has finished sending data

Control Bits in Detail

ECE (ECN-ECHO RFC 3168)

- Routers mark packets rather than dropping them when congestion begins
- A receiver that gets a marked packet sets ECE on its next segment, telling the sender to slow down
- The sender slows down and sets CWR, so the receiver knows the message arrived and can stop sending ECE

ACK

- Set on nearly every segment. When ‘0’, the Acknowledgement number is ignored (first segment of a connection)

SYN and FIN

- SYN = 1 with ACK = 0 is a connection request, since there is nothing yet to acknowledge
- SYN = 1 with ACK = 1 is the reply, which does carry an acknowledgement
- FIN closes the sender’s direction. The other direction stays open until that side sends its own FIN
- Each consumes one sequence number

RST

- Aborts the connection immediately, with no exchange and no guarantee about data in flight
- Sent when a segment arrives for a connection that does not exist, for instance to a port with no listener

PSH

- Asks the receiver to deliver buffered data to the application now rather than waiting for more

URG and the Urgent pointer

- A byte offset from the current sequence number marking where urgent data ends (*Rarely used in practice)*

TCP Window size

The receiver tells the sender how much more data it is willing to accept. This is for flow control.

The field

- 16 bits, so the largest window that can be advertised directly is 65,535 bytes
- It states how many bytes the sender may send, starting at the byte named in the Acknowledgement number field of the same segment
- The value reflects the space available in the receiver's buffer, so it changes as the application consumes data

A window of zero

- Legal, and means: everything up to Acknowledgement number − 1 has been received, but the receiver has not consumed it and wants nothing more for now
- The receiver later reopens the window by sending a segment with the same Acknowledgement number and a non-zero Window size

Why 16 bits is not enough

- On paths with a large bandwidth-delay product, 64 KiB is too small to keep the pipe full
- RFC 7323 defines the Window Scale option, negotiated during the handshake, which multiplies the advertised value by an agreed power of two

TCP Checksum

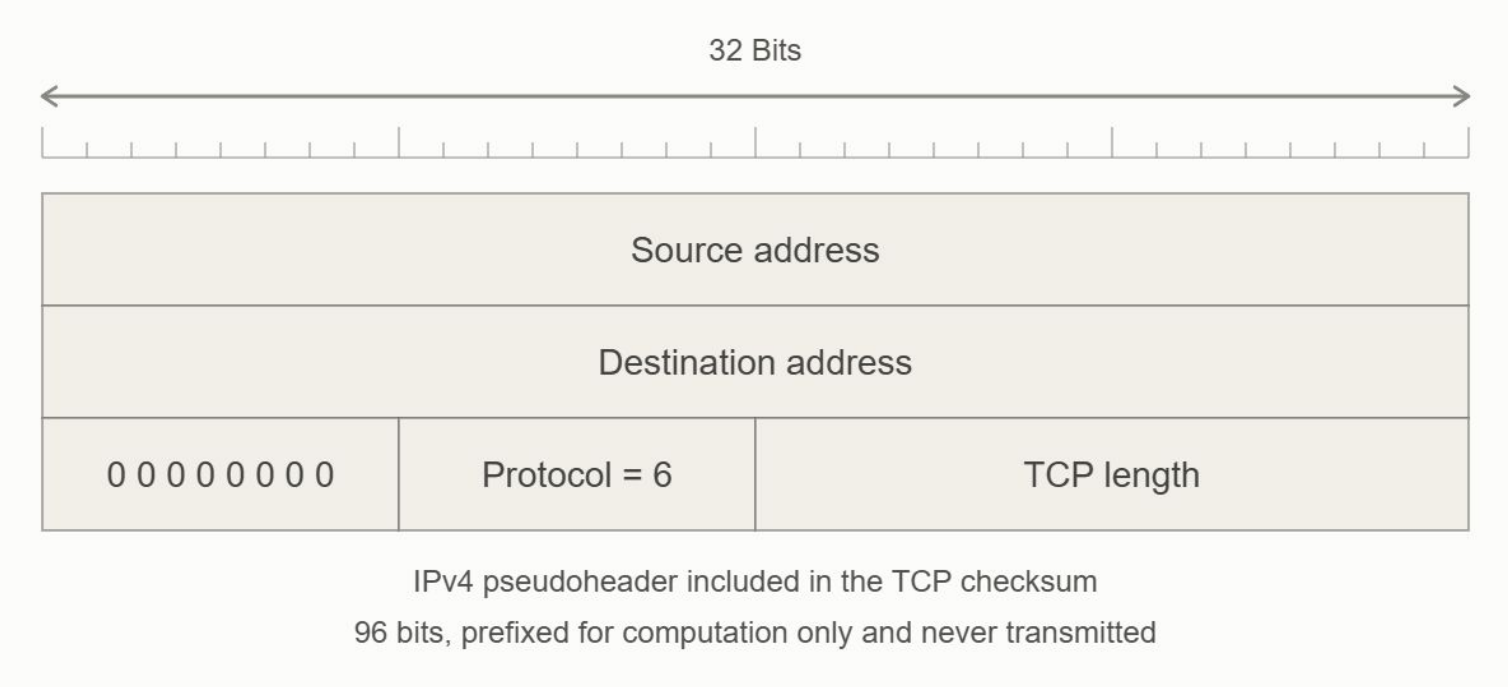
What it covers: The TCP header, The data, A conceptual IPv4 pseudoheader, which is never transmitted

The pseudoheader: Source IP address, destination IP address, a zero byte, the protocol number (6 for TCP), and the TCP segment length.

- Including the IP addresses binds the segment to the endpoints it was addressed to, which detects a mis-delivered segment
- Only fields constant along the path are included, so the TTL and the IP header checksum are excluded

Mandatory

- Unlike UDP over IPv4, a TCP sender must always compute it
- A segment that fails the check is discarded.
- TCP's retransmission then recovers it, so unlike UDP the loss is repaired



IPv4 pseudoheader included in the TCP checksum

96 bits, prefixed for computation only and never transmitted

TCP Header: Options

The options field carries extensions (not in the original header)

Format and limit

- Each option is a kind byte, usually followed by a length byte and its data
- The header length field caps the header at 60 bytes, so options occupy at most 40 bytes

Negotiated at connection setup, carried only on SYN segments

- **Maximum Segment Size:** each side announces the largest segment it is willing to receive. Not a negotiation in the strict sense, since the two directions can differ
- **Window scale:** agrees a power-of-two multiplier for the 16-bit Window size field, lifting the 64 KiB ceiling on paths with a large bandwidth-delay product
- **SACK permitted:** states that this side can handle selective acknowledgement

Carried on every segment once agreed

- **Timestamp:** the sender puts a timestamp in, the receiver echoes it back
  - Gives a round-trip time sample for every segment, which feeds the RTO estimate
  - Also implements **PAWS**, Protection Against Wrapped Sequence numbers: on a fast connection the 32-bit sequence space can wrap while an old segment is still in the network, so a segment arriving with an old timestamp is discarded
- **SACK:** lets a receiver tell the sender exactly which ranges of sequence numbers it has received, so the sender retransmits only what is missing rather than everything from the gap onward

How a TCP Segment is Created

The application writes bytes. TCP decides when a segment leaves and how large it is.

Writes go into the send buffer

- write() deposits bytes in the sender's buffer. It does not cause a transmission
- The buffer holds bytes that are unsent, and bytes that are sent but not yet acknowledged

How much may be sent at this moment

The sender takes the smallest of:

- The bytes available in the buffer
- The usable window: min(CWnd, RWnd) minus the bytes already sent and unacknowledged
- The MSS, which caps a single segment

Challenges in TCP Design

Two things follow from TCP numbering bytes rather than segments, and both complicate the implementation.

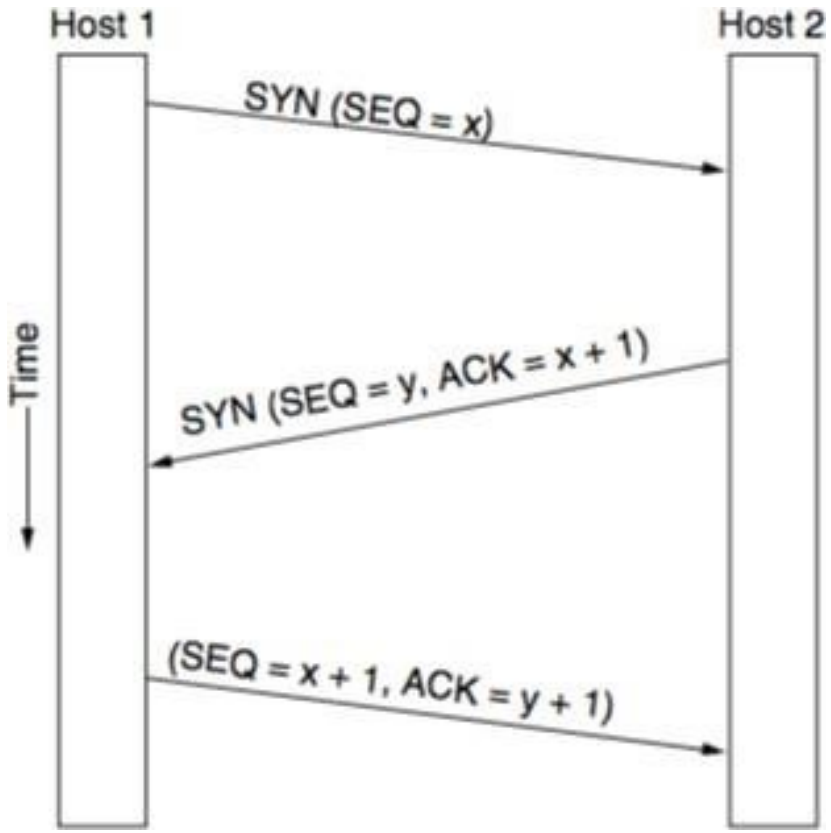
1. A retransmission need not repeat the original segment

- Segments are constructed when they are sent, not fixed in advance
- When a segment is lost, TCP retransmits the missing **bytes**, and may repackage them: merging several small segments into one, or sending fewer bytes if the window has shrunk
- This is possible because acknowledgements name byte positions, not segments. The receiver does not care how the bytes were grouped
- It is also useful: several small segments lost together can be recovered in one larger retransmission

2. Segments may arrive out of order

- The network may reorder packets, and a lost segment leaves a gap while later ones keep arriving
- A receiver that discarded them would force the sender to resend data that had already crossed the network successfully
- So the receiver buffers out-of-order segments and acknowledges only up to the gap

TCP Connection Establishment



The exchange

- Client sends SYN = 1, ACK = 0, SEQ = x
- Server replies SYN = 1, ACK = 1, SEQ = y, ACK number = x + 1
- Client sends SEQ = x + 1, ACK number = y + 1

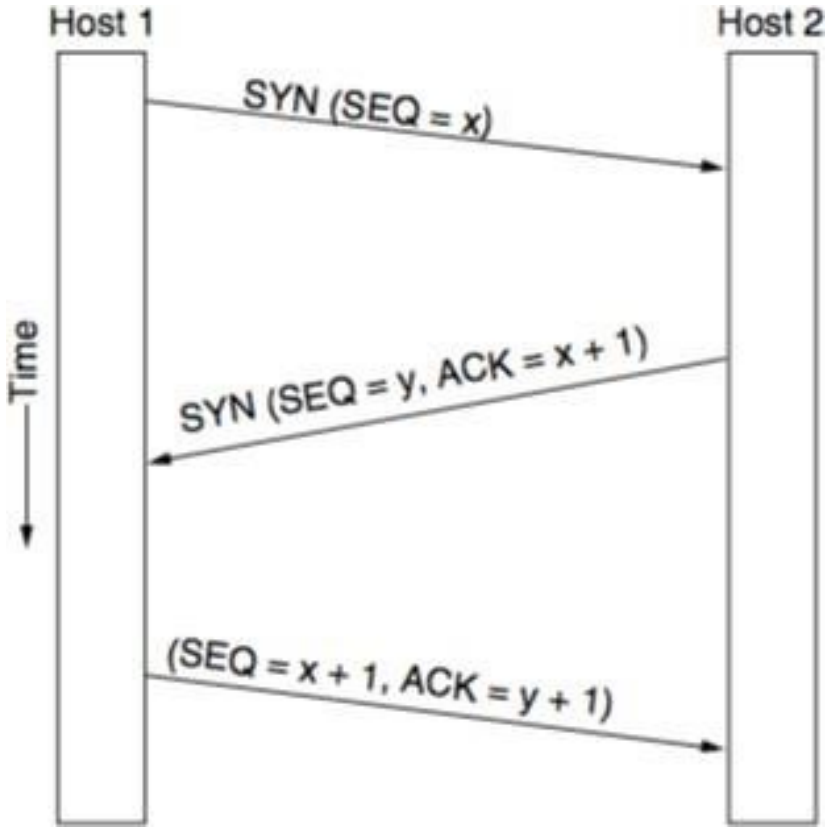
Why x + 1 and not x

- A SYN consumes one sequence number although it carries no data
- So the next byte the client will send is x + 1, and that is what the server acknowledges

States traversed

- Client: CLOSED → SYN SENT → ESTABLISHED
- Server: LISTEN → SYN RCVD → ESTABLISHED
- The server does **not** reach ESTABLISHED on the first message. It commits to the connection only on the third

TCP Connection Establishment



Choosing the initial sequence number

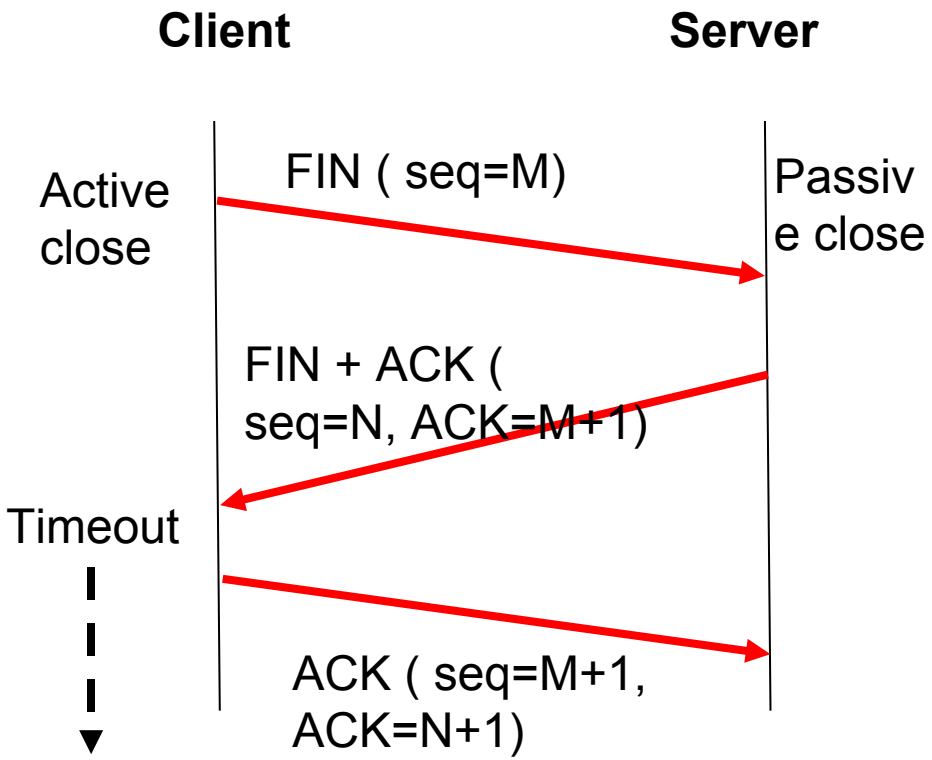
- The ISN must not repeat a value that a still-living segment from an earlier connection could carry
- Original TCP derived it from a clock ticking every 4 microseconds, cycling through 0 to 2^32 − 1, so a full cycle takes about 4.8 hours. This is the Tomlinson clock
- A predictable ISN is also a security problem, because an off-path attacker who can guess it can inject data or a reset into an existing connection. Modern implementations derive the ISN from a cryptographic function of the connection's four-tuple and a secret

SYN flood

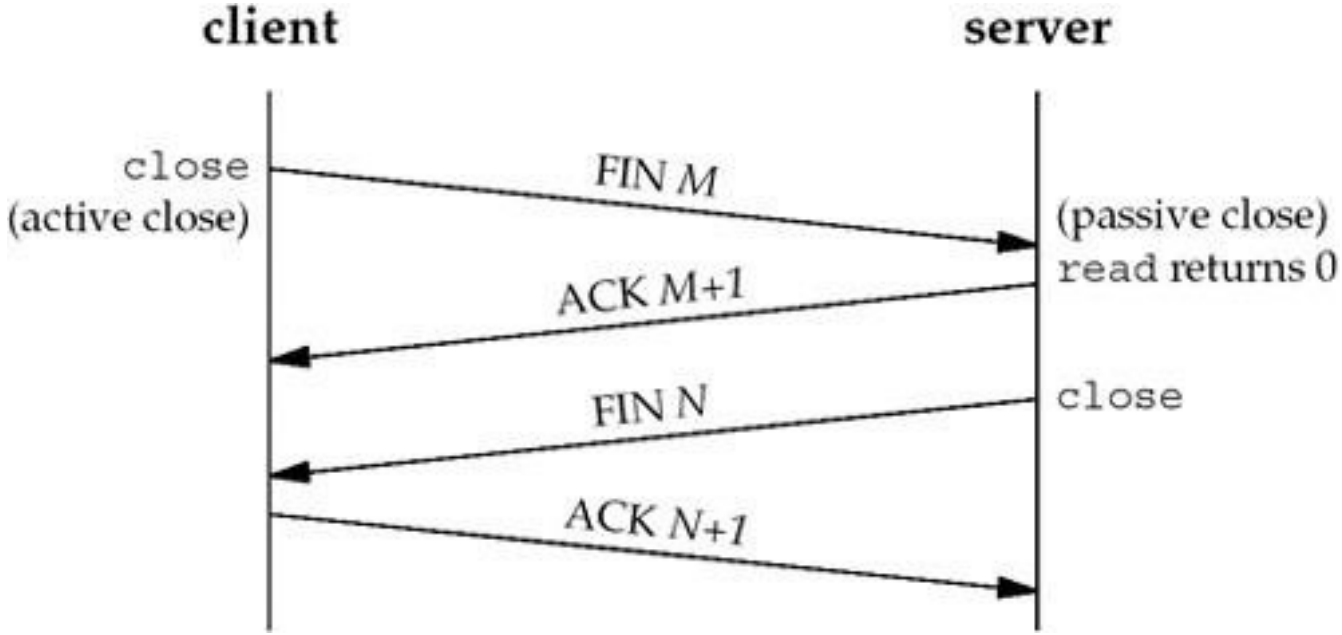
- The handshake establishes correctness, not resource safety. The server allocates a control block at SYN RCVD, before any confirmation
- An attacker sending many SYNs that are never completed exhausts that table

SYN cookies: the server encodes the state it needs into the ISN it sends, keeps nothing, and reconstructs the connection when the third message returns with that value echoed back

TCP Connection Release

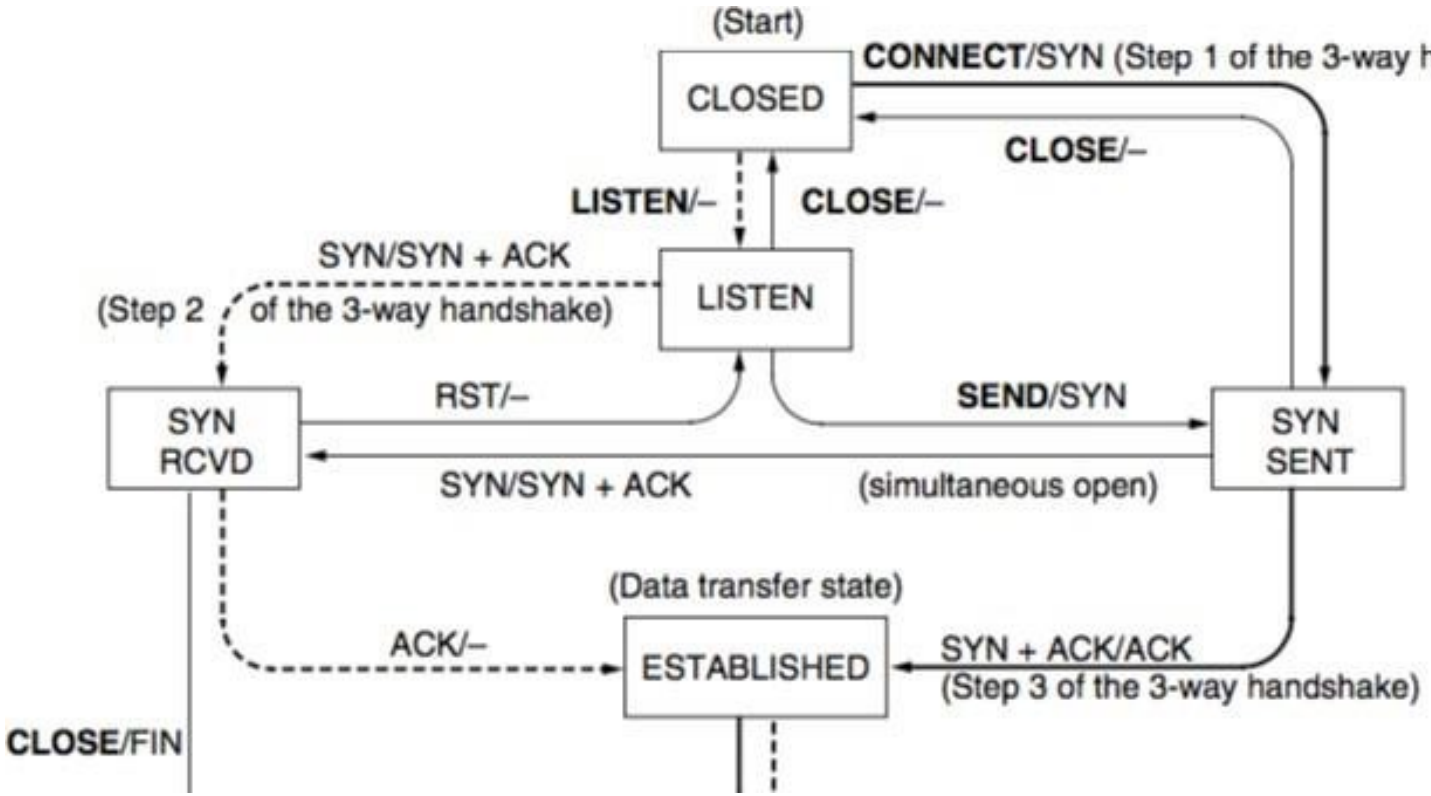


Case - I



Case - II

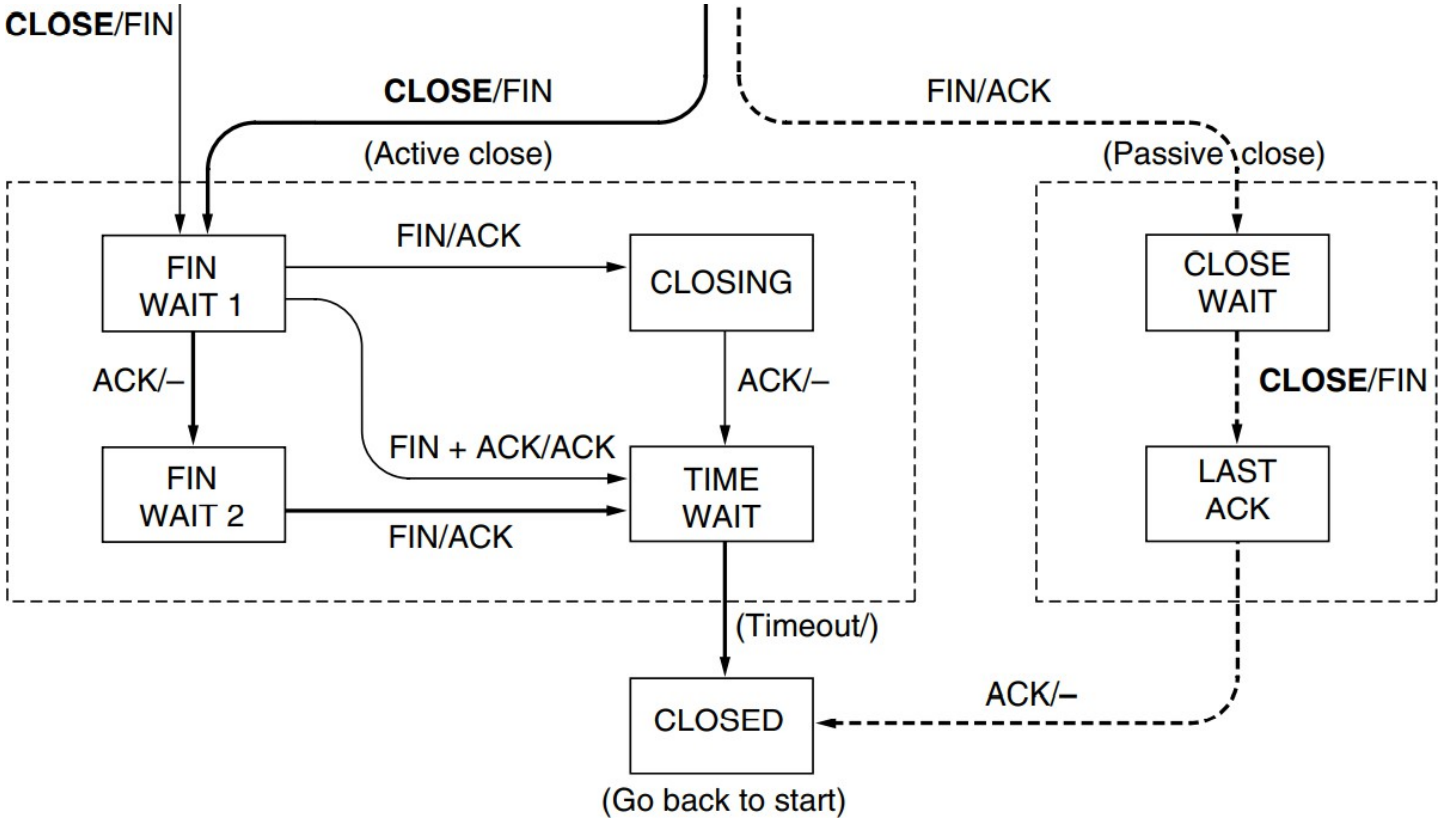
TCP State Transition Diagram – Connection Modelling



State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

Event/Action
---- Server
Client

TCP State Transition Diagram – Connection Modelling



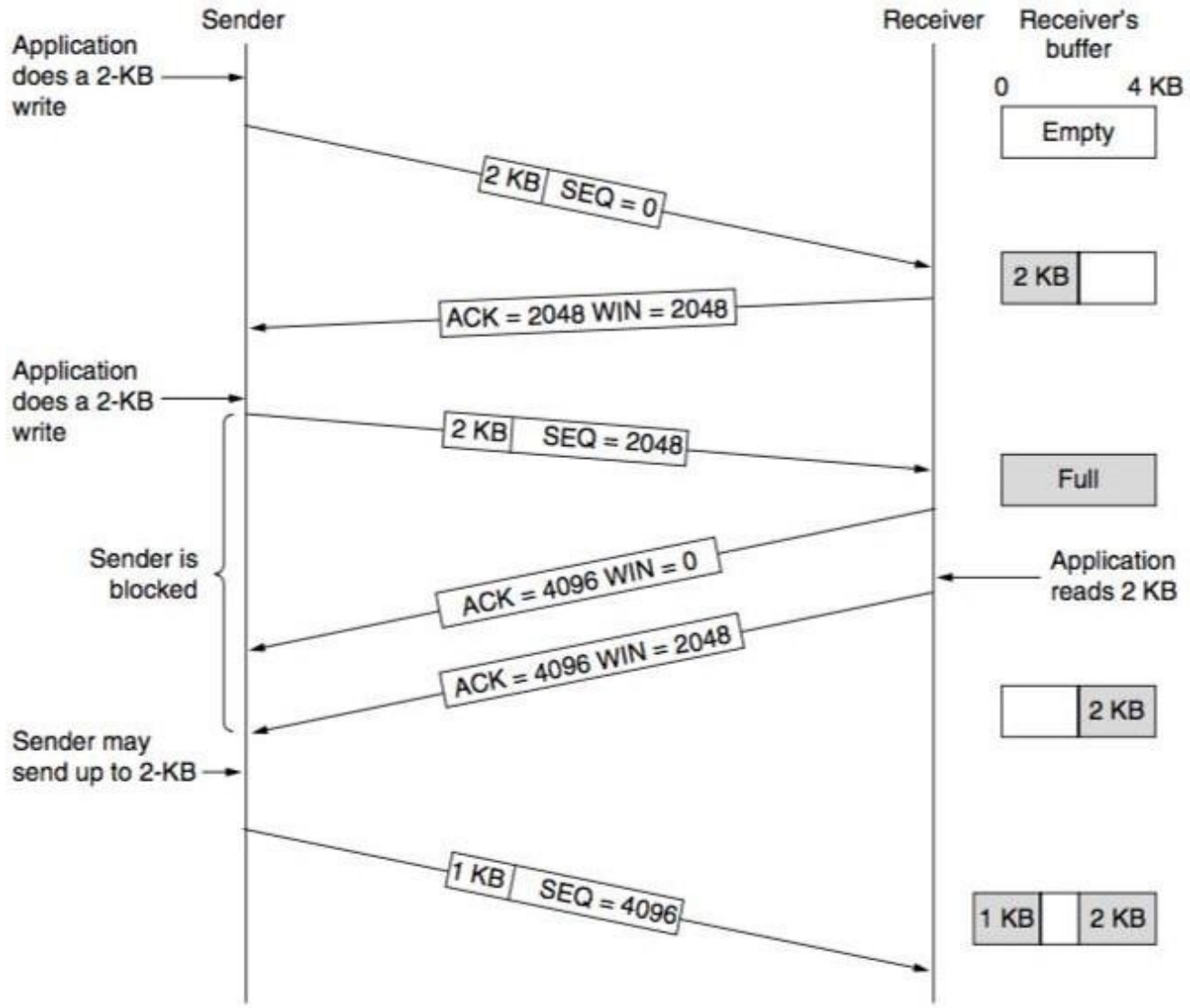
State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

Event/Action
---- Server
Client

TCP Sliding Window

Flow control rationale:

The receiver should advertise the available buffer space (receiver window size) to the sender so that to restrict it from sending too much data beyond the receiver's capacity.



TCP Sliding Window

A zero window stops the sender but two exceptions exist:

The deadlock:

- The receiver advertises WIN = 0, and the sender stops
- Later the application consumes data and the receiver sends a window update
- **That update is pure acknowledgement, carrying no data, so it is not retransmitted if lost**
- The receiver believes it has reopened the window. The sender is still waiting. Neither side will act
- This is the “potential deadlock”

Exception 1: the window probe

- The sender may send a one-byte segment even with the window closed
- The receiver must respond with its current acknowledgement number and window size, so the lost update is effectively recovered
- Probing zero windows is required, not optional
- The responsibility sits with the sender, deliberately: the side that is blocked is the side that must break the deadlock

Exception 2: urgent data

- May be sent regardless of the window, for example to let a user kill a runaway process on the remote machine
- Rarely used in practice, and applications are discouraged from relying on it

Delayed Acknowledgments

A receiver need not acknowledge immediately. Waiting briefly lets several things travel in one segment.

- Consider a telnet connection, that reacts on every keystroke.
- In the worst case, whenever a character arrives at the sending TCP entity, TCP creates a 21 byte TCP segment, 20 bytes of header and 1 byte of data. For this segment, another ACK along with window update is sent when the application reads that 1 byte. This results in a huge wastage of bandwidth.

The mechanism

- The receiver holds its acknowledgement for a short time, hoping more data arrives, or that its own application produces a reply to carry it on
- RFC 1122: the delay MUST be less than 0.5 seconds, and in a stream of full-sized segments there SHOULD be an acknowledgement for at least every second segment

However, the sender can still send multiple short data segments**. Inefficient!!**

Nagle's Algorithm

- When data come into the sender in small pieces, just send the first piece and buffer all the rest until the first piece is acknowledged.

- Then send all buffered data in one TCP segment and start buffering again until the next segment is acknowledged.
  - **Only one short packet can be outstanding at any time.**

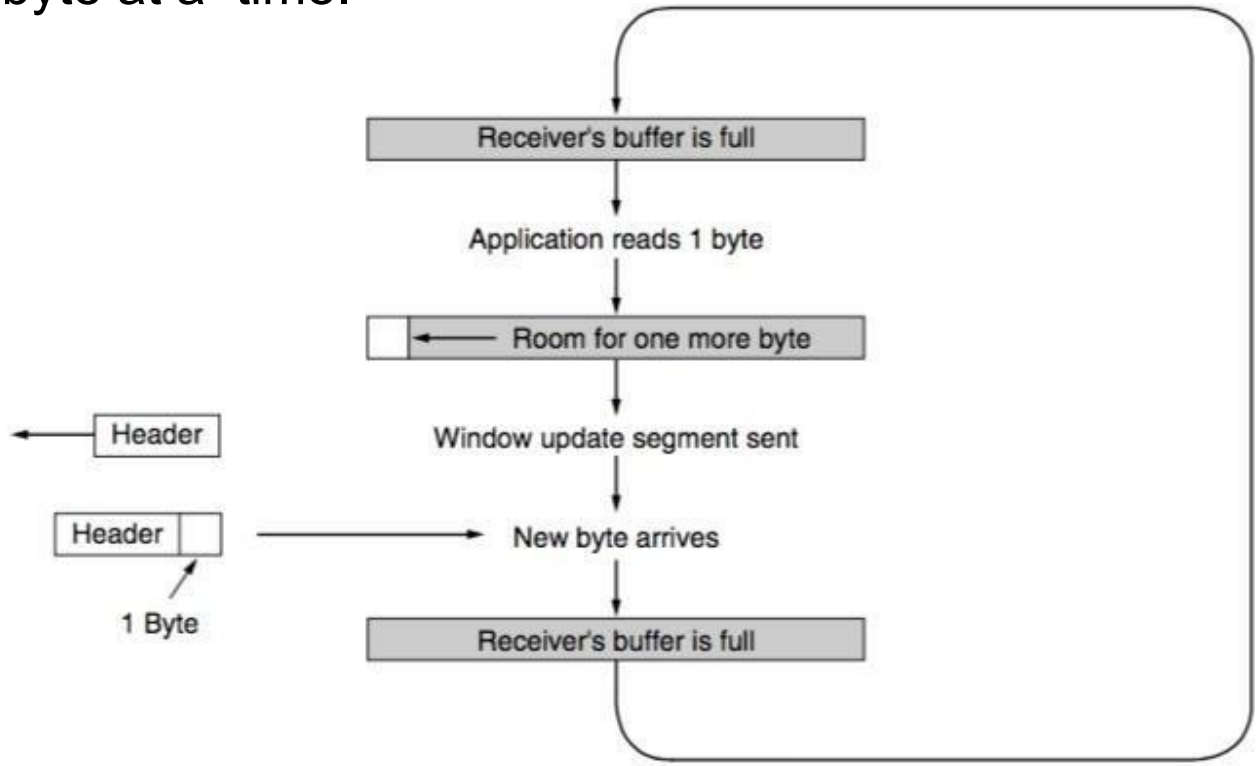
- **Nagle's Algorithm and Delayed Acknowledgement**
  - Receiver waits for data and sender waits for acknowledgement – results in a stall
  - **Do we want Nagle's Algorithm all the time?**

- No. Interactive applications that send small messages and need low latency turn it off, using the TCP_NODELAY socket option
- Examples: remote login, mouse and cursor updates, anything issuing many small requests and waiting for each reply

Silly Window Syndrome

- The receiver can also cause small segments, by advertising tiny windows. Clark's solution stops it.
- Data are passed to the sending TCP entity in large blocks, but an interactive application on the receiver side reads data only 1 byte at a time.

Clark's solution: Do not send window update for 1 byte.
Wait until sufficient space is available at the receiver buffer.



Handling Short Segments

Three mechanisms, each preventing small segments from a different cause. All 3 work by waiting briefly.

Mechanism	Acts at	Cause it addresses	What it does
Delayed acknowledgement	Receiver	Acknowledgement, window update and reply sent as three separate small segments	Holds the acknowledgement briefly so they combine into one
Nagle's algorithm	Sender	The sending application delivers data to TCP a byte at a time	Buffers until the outstanding data is acknowledged
Clark's solution	Receiver	The receiving application fetches data from TCP a byte at a time	Does not advertise a window until worthwhile space is free

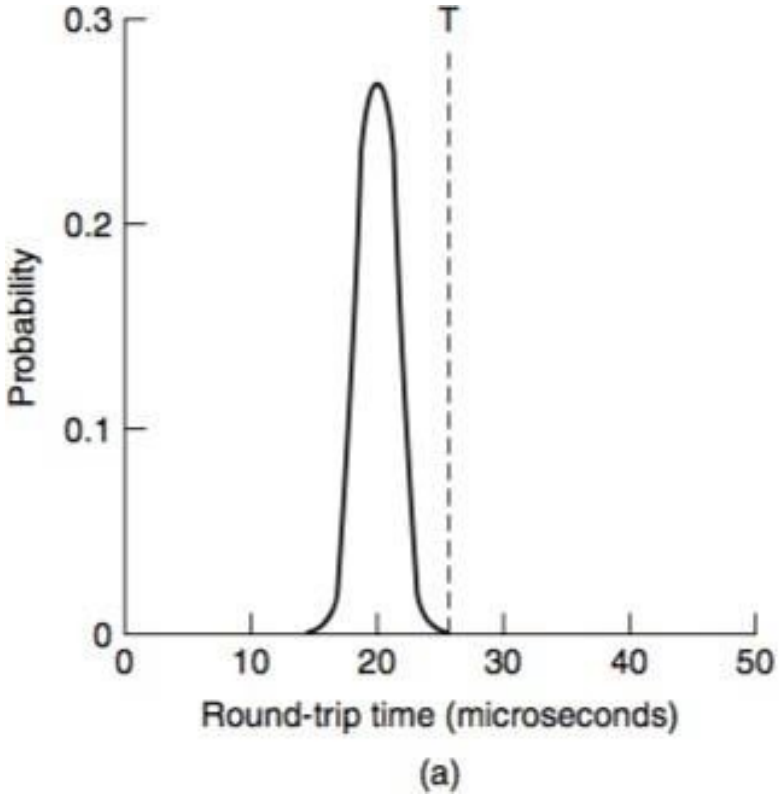
Handling Out of Order inTCP

- TCP buffers out of order segments and forward a duplicate acknowledgement to the sender.
- **Acknowledgement in TCP is Cumulative acknowledgement**
- Receiver has received bytes 0, 1, 2, _, 4, 5, 6, 7
  - TCP sends a cumulative acknowledgement with ACK number 3, acknowledging everything up to byte 2
  - Once 4 is received, a duplicate ACK with ACK number 3 (next expected byte) is forwarded
  - After timeout, sender retransmits byte 3
  - Once byte 3 is received, it can send another cumulative ACK with ACK number 8 (next expected byte)

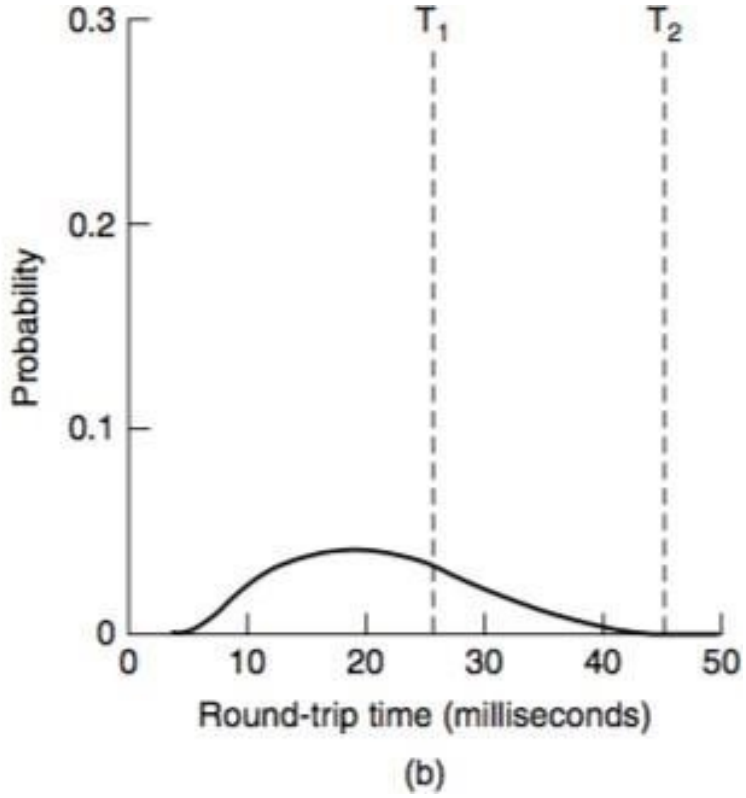
TCP Timer Management: the Retransmission Timeout

- **TCP Retransmission Timeout (RTO)**: When a segment is sent, a retransmission timer is started
  - **If the segment is acknowledged before the timer expires, the timer is stopped**
  - **If the timer expires before the acknowledgement comes, the segment is retransmitted**
- **What can be an ideal value of RTO ? (Too short/Too Long)**
- **Possible solution**: Estimate RTT, and RTO is some positive multiples of RTT
- **RTT estimation is difficult for transport layer – why?**

RTT at Data Link Layer vs. RTT at Transport Layer



Data Link Layer



Transport Layer

RTT Estimation at Transport Layer

- Use a dynamic algorithm that constantly adapts the timeout interval, based on continuous measurements of network performance.
- **Jacobson's algorithm (1988) - used in TCP**
  - For each connection, TCP maintains a variable, ***SRTT (smoothed Round Trip Time)*** – best current estimate of the round trip time to the destination
  - When a segment is sent, a timer is started (both to see how long the acknowledgement takes and also to trigger a retransmission if it takes too long)
  - If the ACK gets back – measure the time (say, *R*)
  - Update SRTT as follows

$$SRTT = \alpha\, SRTT + (1 - \alpha)\, R$$ **(Exponentially Weighted Moving Average – EWMA)**

α is a smoothing factor that determines how quickly the old values are forgotten.
Typically α = 7/8

Problem with EWMA

- Even given a good value of SRTT, choosing a suitable RTO is non-trivial.
- Initial implementation of TCP used RTO = 2SRTT, but experience showed that a constant value was too inflexible, because it failed to respond when the **variance went up (RTT fluctuation is high) – happens normally at high load.**
- **Therefore, Jacobson proposed considering variance of RTT during RTO estimation.**

RTO Estimation

Update RTT variation (*RTTVAR*) as follows.

$$RTTVAR = \beta\, RTTVAR + (1 - \beta) \cdot |SRTT - R|$$

Typically β = ¾

RTO is estimated as follows,

$$RTO = SRTT + 4 \times RTTVAR$$

- **Why 4 ?**
  - **Somehow arbitrary but multiplication by 4 can be done with a single shift, and less than 1% of all packets come in more than four standard deviations late.**

Karn's Algorithm

A retransmitted segment produces an acknowledgement that cannot be timed, because it is impossible to tell which transmission it answers.

The ambiguity

- A segment is sent, the timer expires, and it is retransmitted. An acknowledgement then arrives
- It may be acknowledging the original, which was merely slow, or the retransmission
- There is no way to tell, because both carried the same sequence numbers

Why guessing is worse than not measuring

- If it answers the slow original but is timed from the retransmission, the sample is far too small, and the RTO shrinks, causing more spurious retransmissions and more ambiguous samples
- If the acknowledgement answers the retransmission but is timed from the original, the sample is far too large, and the RTO inflates
- Either way the estimate degrades at exactly the moment the connection is in trouble

Karn's Algorithm

Karn's rule, two parts

1. Do not update SRTT or RTTVAR from any segment that has been retransmitted
2. On each retransmission, double the RTO. Keep the doubled value until a segment is acknowledged on its first transmission

Why the second part is necessary

- Without it the first part would freeze the timeout entirely. If every segment is being retransmitted, no samples are taken, and the RTO stays at a value that is evidently too small
- The doubling provides an estimate-free way to back off, and it recovers as soon as one clean sample becomes available

The modern exception

- With the timestamp option, the acknowledgement echoes the timestamp of the transmission it is actually answering
- The ambiguity disappears, so samples can be taken even from retransmitted segments

Other TCP Timers

- **Persistent TCP Timer:** Avoid deadlock when receiver buffer is announced as zero
  - **After the timer goes off, sender forwards a probe packet to the receiver to get the updated window size**

- **Keepalive Timer:** Close the connection when a connection has been idle for a long duration

- **TCP TIME_WAIT:** It runs for twice the maximum packet lifetime to make sure that when a connection is closed, all packets created by it have died off.

1986 Congestion Collapse

In October 1986 the Internet had the first of what became a series of congestion collapses.

- Throughput from Lawrence Berkeley Laboratory (LBL) to UC Berkeley, two sites 400 yards apart, dropped from 32 Kbps to 40 bps (dropped by a factor of a thousand)

Why a loaded network collapses instead of degrading gracefully:

- Queues build, so round trip times rise
- Timeouts fire on segments that were only delayed, and senders retransmit
- The retransmissions add load, which lengthens queues, which causes more timeouts
- The network stays busy carrying duplicates of data it has already delivered

The links stay saturated while goodput collapses. The network is busy, but much of what it carries is duplicates, or packets that will be dropped before they arrive.

Jacobson's Response, 1988

Early TCP congestion control came from Van Jacobson and Michael Karels, after the collapse:

The constraint: it had to be instantly deployable. No change to the protocol, no new header fields, no cooperation from routers.

Jacobson describes the implementation as one new state variable and three lines of code in the sender:

- Add a congestion window, CWnd, to the per connection state
- On starting, or restarting after a loss, set CWnd to one packet
- On each acknowledgement for new data, increase CWnd by one packet
- When sending, send the minimum of the receiver's window and CWnd

Packet loss as the congestion signal:

- The sender cannot see queues inside the network, so it needs something observable
- Packets can be damaged in transit, but that is rare, so a lost packet almost always means a full queue
- Loss is therefore a usable binary signal: congestion, or no congestion

TCP Congestion Window

CWnd is the number of bytes the sender may have in the network at any time.

- Maintained by the sender alone. No header field carries it and the receiver never sees it
- Contrast RWnd, the receive window, which the receiver advertises in every segment and which states the free space currently in its buffer

Two separate limits, and the sender obeys whichever is tighter:

- RWnd is: how much can the receiver absorb?
- CWnd is : how much can the network absorb?

Sender Window (SWnd) = min (CWnd, RWnd)

For example:

- The receiver says send 64 KB, but the sender knows that bursts over 32 KB clog the network. It sends 32 KB
- The receiver says send 64 KB, and the sender knows that bursts of up to 128 KB get through effortlessly. It sends the full 64 KB

Sending rate = SWnd / RTT, since one window is sent per round trip. The window is the rate control.

AIMD on the Congestion Window

TCP implements AIMD using a window, with packet loss as the binary signal.

Additive Increase Multiplicative Decrease, Chiu and Jain (1989).

Let *w(t)* be the sending rate. *a (a > 0)* is the additive increase factor, and *b(0<b<1)* is the multiplicative decrease factor

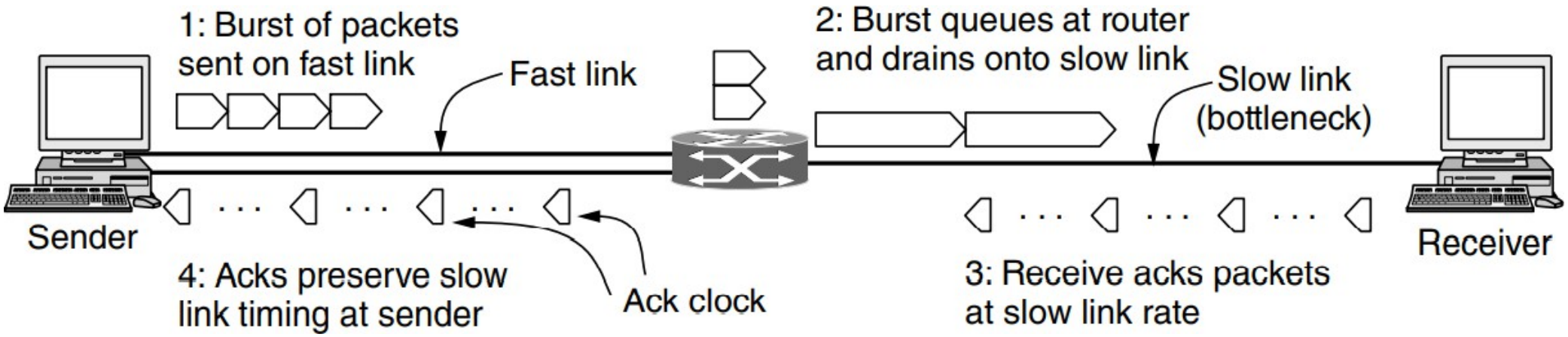
$$w(t+1) = \begin{cases} w(t) + a & \text{if congestion is not detected} \\ w(t) \times b & \text{if congestion is detected} \end{cases}$$

TCP applies this to CWnd rather than to a rate directly, because the window is the only quantity the sender controls. The rate follows from it.

TCP applies both factors per acknowledgement rather than once per round trip. Why?

Adjust CWnd based on ACK Clock

- CWnd says how many packets may be in the network, not when to send them. A sender that releases its whole window at once obeys the limit and still overflows queues. Acknowledgements are what space the packets out.



- **ACK returns to the sender at about the rate that packets can be sent over the slowest link in the path.**
- **Trigger CWnd adjustment based on the rate at which ACK are received.**
- **By using an ack clock, TCP smoothes out traffic and avoids unnecessary queues at routers.**

Increase Rate Exponentially at the Beginning – The Slow Start

- AIMD rule will take a very long time to reach a good operating point on fast networks if the CWnd is started from a small size.

- Consider a 10 Mbps link with 100 ms RTT:
- Bandwidth delay product = 10 Mbps x 100 ms = 1 Mbit
- At 1250 bytes per packet, that is 100 packets in flight to fill the pipe
- CWnd starts at 1 packet. Additive increase adds 1 packet per RTT, so reaching 100 packets takes about 100 RTTs, which is 10 seconds.

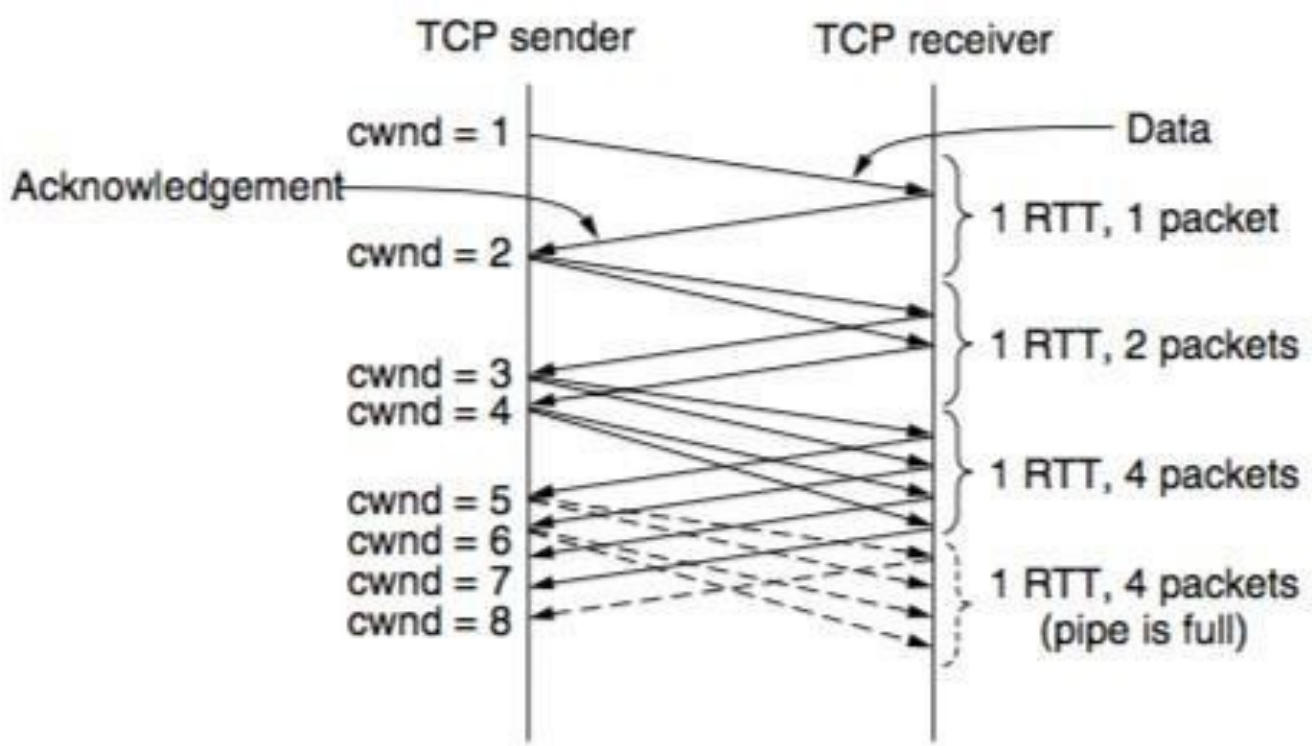
- **Slow start:** double CWnd every RTT instead. The same 100 packets is reached in about 7 RTTs, under a second.

- **Slow Start - Exponential increase of rate to avoid slow convergence**
  - **Rate is not slow at all !**

TCP Slow Start

- Every ACK segment allows two more segments to be sent

- For each segment that is acknowledged before the retransmission timer goes off, the sender adds one segment's worth of bytes to the congestion window.



Slow Start Threshold

- Slow start causes exponential growth, eventually it will send too many packets into the network too quickly.

- To keep slow start under control, the sender keeps a threshold for the connection called the **slow start threshold (ssthresh).**

- Initially ssthresh is set to BDP (or arbitrarily high), the maximum that a flow can push to the network.

- Whenever a packet loss is detected by a RTO, the ssthresh is set to be half of the congestion window

Additive Increase (Congestion Avoidance)

- Whenever ssthresh is crossed, TCP switches from slow start to additive increase.

- Usually implemented with an partial increase for every segment that is acknowledged, rather than an increase of one segment per RTT.

- A common approximation is to increase Cwnd for additive increase as follows:

$$CWnd = Cwnd + \frac{MSS \times MSS}{CWnd}$$

Detecting Loss: Timeout and Duplicate ACKs

- Two ways to trigger a congestion notification in TCP: 1. RTO (2) Duplicate ACK
- **RTO**: A sure indication of congestion, however it is time consuming. The RTO is SRTT plus four times RTTVAR, so it is always longer than a round trip, and the connection sends nothing while it waits
- **Duplicate ACK:** Receiver sends a duplicate ACK when it receives out of order segment
  - **A loose way of indicating congestion**
  - RFC 5681: a TCP receiver should send an immediate duplicate ACK when an out of order segment arrives
  - Each duplicate therefore means a later segment did arrive, and the gap is still unfilled
  - **TCP arbitrarily assumes that THREE duplicate ACKs (DUPACKs) imply that a packet has been lost**

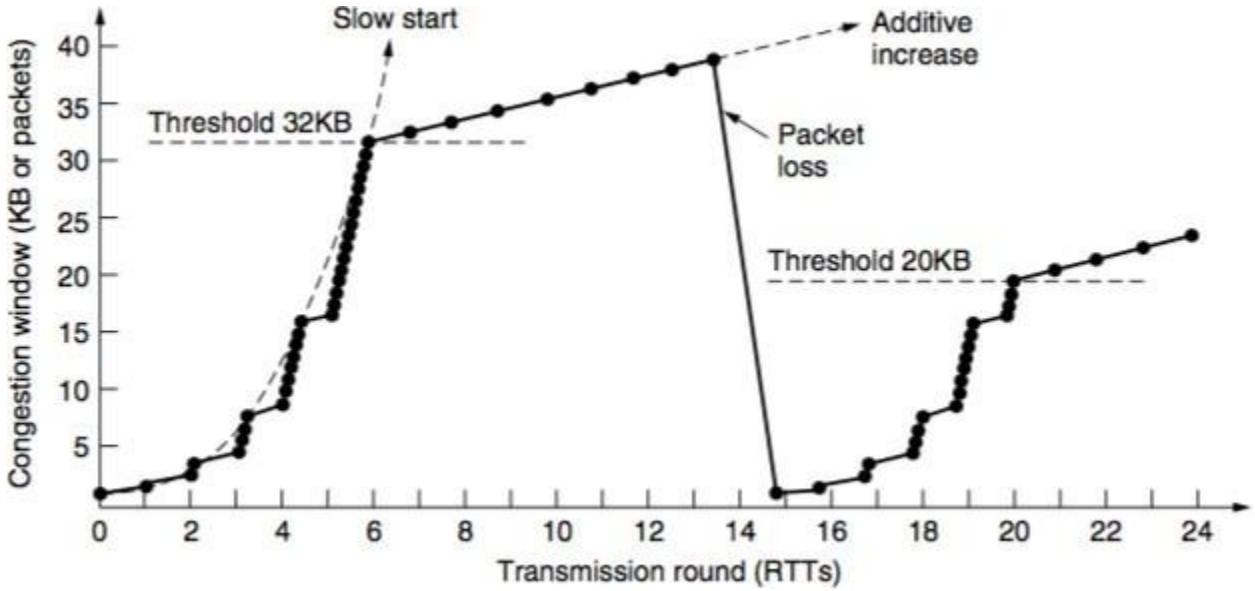
    – triggers congestion control mechanism
  - The identity of the lost packet can be inferred – **the very next packet in sequence**
  - **Retransmit the lost packet and trigger congestion control**

Fast Retransmission – TCP Tahoe

Use THREE DUPACK as the sign of congestion

Once 3 DUPACKs have been received,

- Retransmit the lost packet (**fast retransmission)** – takes one RTT
- Set ssthresh as half of the current CWnd
- Set CWnd to 1 MSS



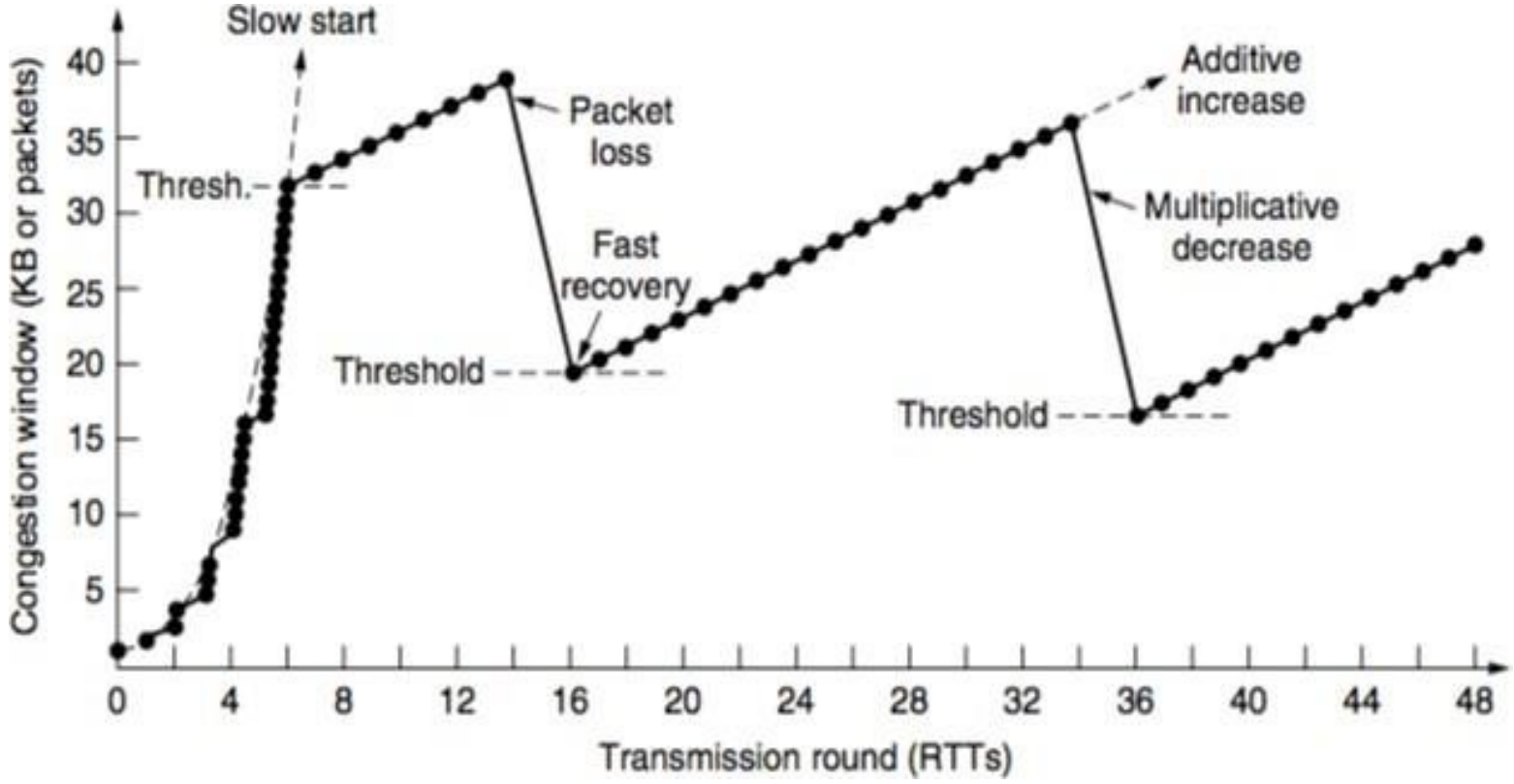
Fast Recovery - TCP Reno

- **Once a congestion is detected through 3 DUPACKs, do TCP really need to set CWnd = 1 MSS ?**

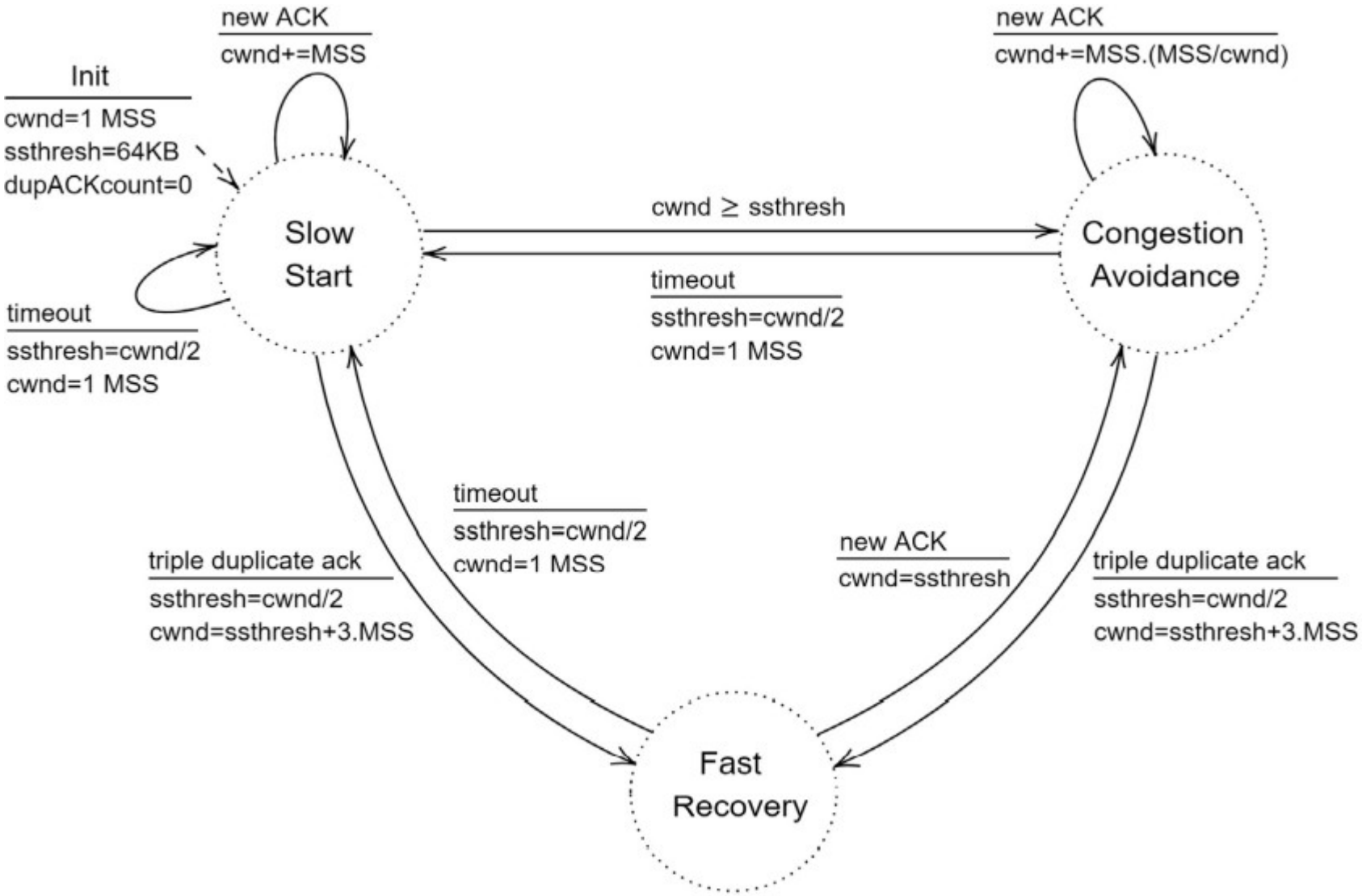
- DUPACK means that **some segments are still flowing in the network**– a signal for temporary congestion, but not a prolonged one

- Immediately transmit the lost segment (**fast retransmit)**, then transmit additional segments based on the DUPACKs received **(fast recovery)**

Fast Recovery - TCP Reno

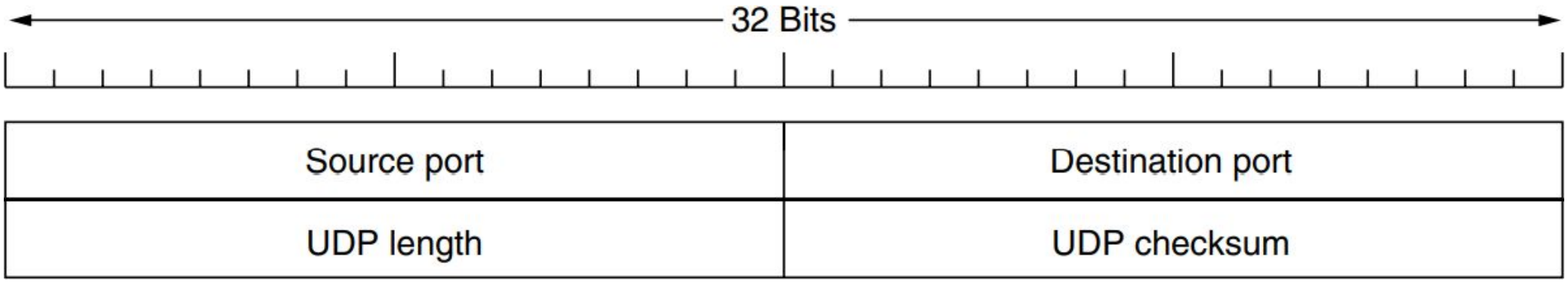


TCP Protocols Summarized



UDP : User Datagram Protocol

- UDP adds the minimum to IP: a way to reach a process rather than a host, and an optional integrity check.
    - •Connectionless. No handshake, no state, no release
    - •Transmits segments consisting of an 8-byte header followed by the payload
    - •Applications send encapsulated IP datagrams without establishing a connection
    - •No flow control, no congestion control, no retransmission, no reordering
    - •A damaged segment is discarded, not repaired



UDP Header

UDP : User Datagram Protocol

- Four fields, 16 bits each, 8 bytes in total.

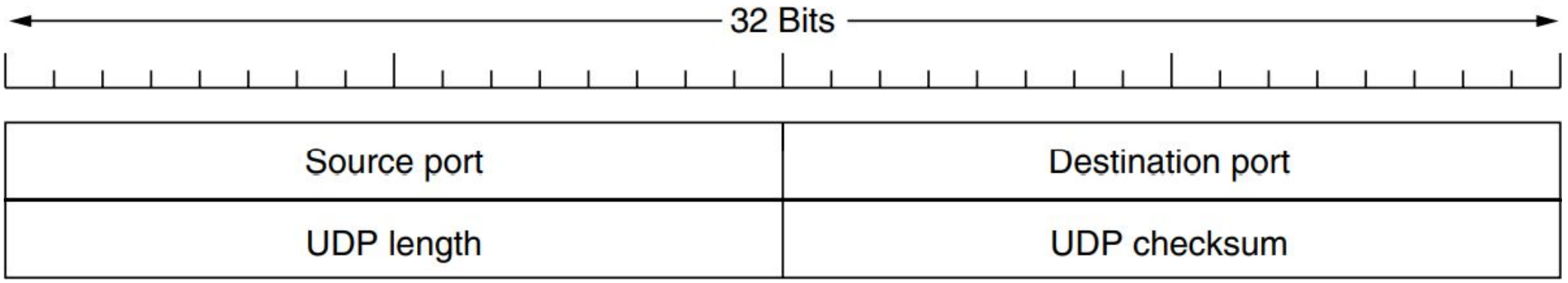
  - **Source port, destination port:** identify the sending and receiving processes
  - **UDP length:** header plus data
  - **UDP checksum:** optional in IPv4, mandatory in IPv6

  Smallest datagram

  - 8 bytes, the header alone, carrying no data

  Largest datagram

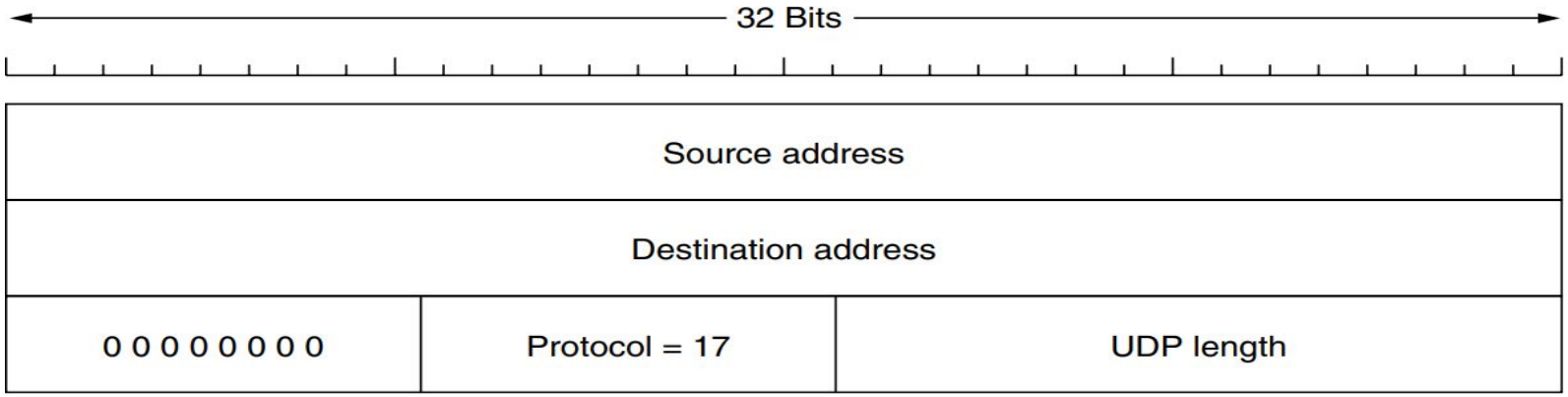
  - The IPv4 total length field is 16 bits, so a whole IP packet is at most 65,535 bytes
  - The IPv4 header takes 20 of these, leaving 65,515 bytes
  - So UDP length can be at most 65,515, even though a 16-bit field could hold 65,535
  - The UDP header takes 8 of these, so the largest payload is 65,507 bytes



UDP Header

UDP Checksum

- The only error protection UDP offers, and in IPv4 it is optional.



IPV4 Pseudoheader included in UDP checksum

What it covers: The UDP header, The data, A conceptual IPv4 Pseudoheader (which is never transmitted)

The Pseudoheader:

- Only fields that are constant along the path are included. The TTL and the IP header checksum change at every hop, so they are left out
- Including the IP addresses binds the segment to the endpoints it was addressed to, which gives protection against misrouted datagrams

Optional in IPv4

- An all-zero transmitted checksum means the sender computed none
- If the computed value happens to be zero, it is transmitted as all ones, so the two cases stay distinguishable
- Mandatory in IPv6, where a zero checksum must be rejected by the receiver

Detection only

A segment that fails the check is discarded, with no notification to sender or receiver

How UDP Delivers Datagrams?

- UDP's delivery model is defined by two properties:

Demultiplexing: which socket gets the datagram

- UDP selects the receiving socket using the destination IP address and destination port
- One socket therefore receives datagrams from many different senders
- Each datagram is delivered with the sender's address and port, so the application can tell senders apart. UDP keeps no state about them.

Message boundaries are preserved

- One send produces one datagram
- One receive returns exactly that datagram, whole
- The unit the application wrote is the unit the application reads

UDP suits exchanges where the message is the natural unit: a DNS query, a sensor reading, a single video frame

UDP vs TCP Comparison

	UDP	TCP
Connection	None. Each datagram is independent	Handshake before data, release afterwards
State at the endpoints	None	A control block per connection: sequence numbers, windows, timers, state machine
Header	8 bytes, fixed	20 bytes, up to 60 with options
Demultiplexing	Destination IP and port	All four: source IP, source port, destination IP, destination port
Unit delivered	One send gives one receive, whole	A byte stream. Boundaries are not preserved
Checksum	Optional in IPv4, mandatory in IPv6	Mandatory

UDP vs TCP Comparison

	UDP	TCP
Lost segments	Not detected, not repaired	Detected and retransmitted
Out of order arrival	Delivered as it arrives	Restored to order before delivery
Duplicates	Delivered twice	Suppressed
Flow control	None	Receive window
Congestion control	None	AIMD on the congestion window
Receivers	One, many (multicast), or all (broadcast)	Exactly two endpoints